

Machine Learning

Lecture 14: Ensemble learning

王浩

信息科学与技术学院

Email: wanghao1@shanghaitech.edu.cn

Ensemble methods

- ▶ Why learn one classifier when you can learn many?
- ▶ Ensemble: combine many predictors
 - combinations of predictors
 - may be same type of learner or different



集成学习：动机

- 在机器学习中，直接建立一个高性能的分类器是很困难的。
- 但是，如果能找到一系列性能较差的分类器，并把它们集成起来的话，也许就能得到更好的分类器。

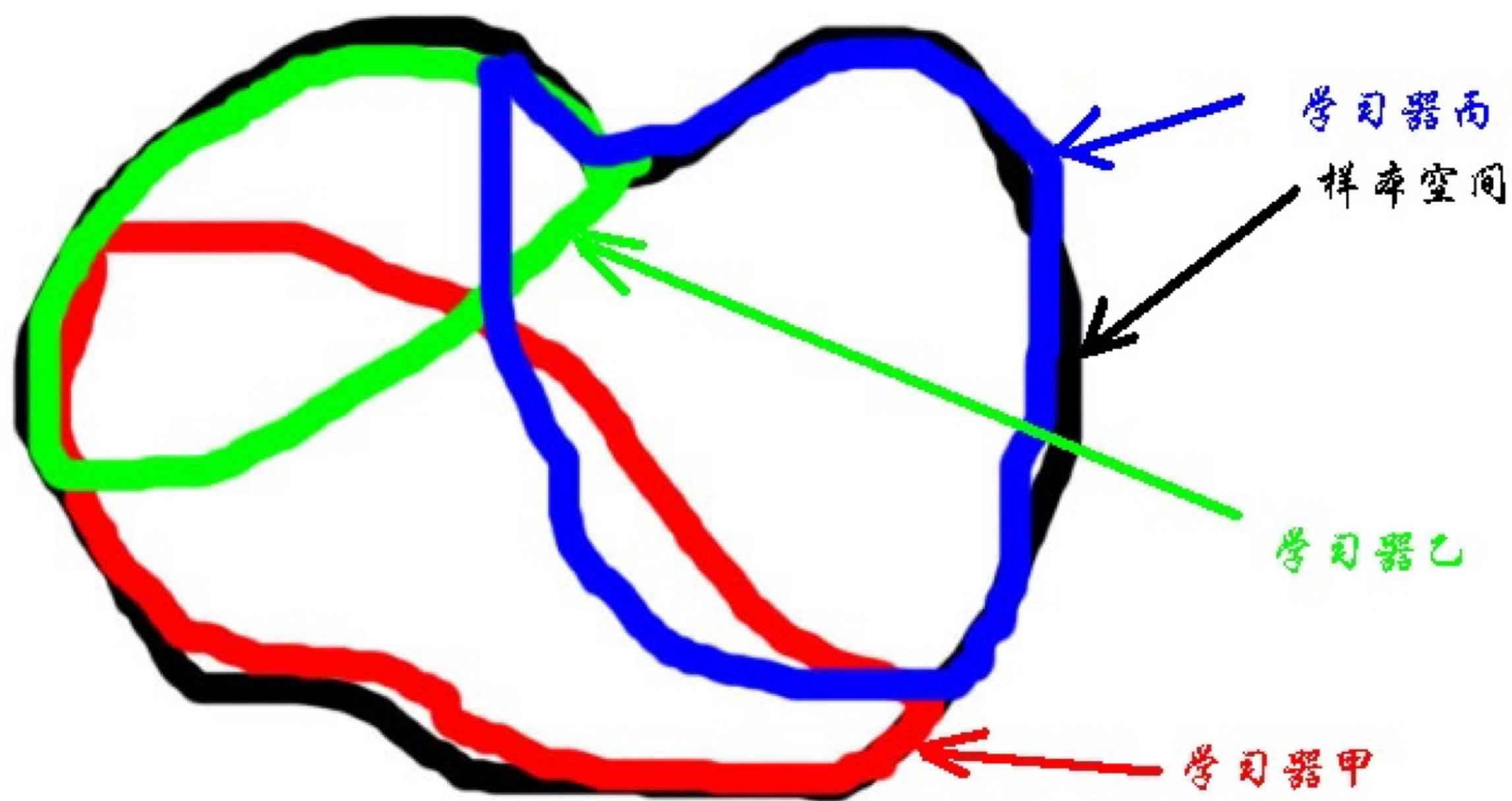
三个臭皮匠，胜过诸葛亮

集成学习：动机

- 集成学习，就是一种把输入送入多个学习器，再通过某种办法把学习的结果集成起来的办法。
- 这每一个学习器，也就相应的被称为“弱学习器”（仅仅强于随机猜测）。
- 几个弱学习器集成在一起之后，可以变成强学习器（预测效果更强）

弱学习器就是臭皮匠，强学习器就是诸葛亮

本质：用多个学习器覆盖样本空间



集成学习实际上代表了一种与传统不同的思维理念。

传统的机器学习一般认为是单模型的，对于模型的分析总是在整体上完成。

集成学习

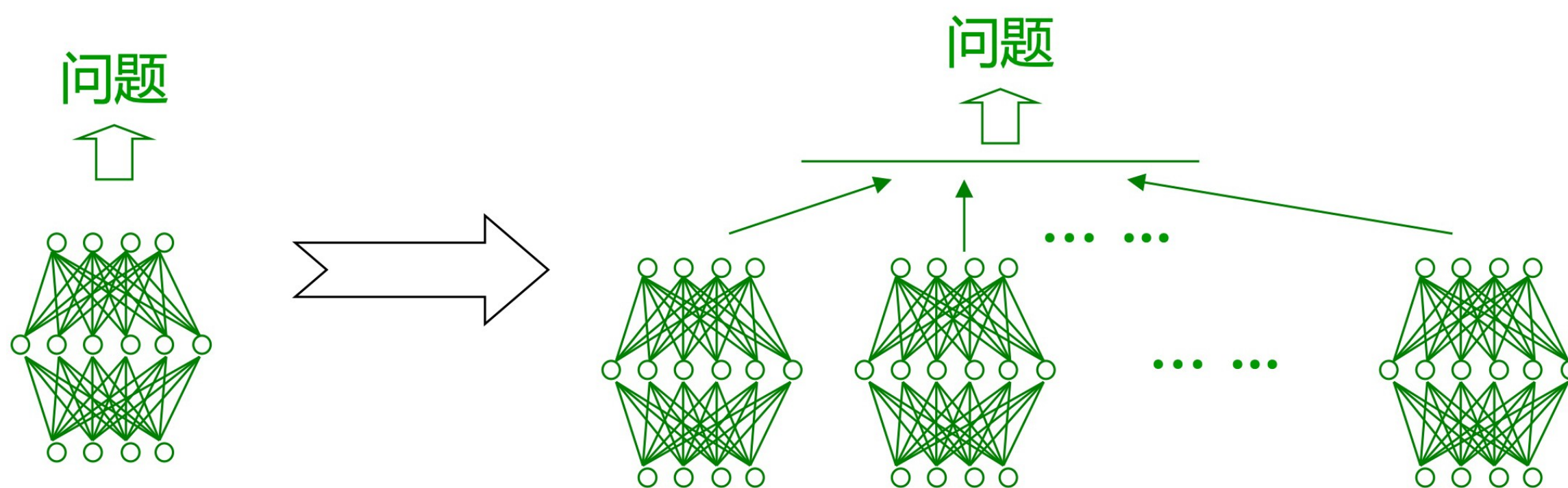
集成学习（Ensemble Learning）是一种机器学习范式，它使用多个（通常是同类型的）学习器来解决同一个问题

集成学习中使用的多个学习器称为个体学习器

当个体学习器均为决策树时，称为“决策树集成”

当个体学习器均为神经网络时，称为“神经网络集成”

.....



集成学习的重要性

问题：对20维超立方体空间中的区域分类

左图中纵轴为错误率

从上到下的四条线分别表示：

平均神经网络错误率

最好神经网络错误率

两种神经网络集成的错误率

令人惊奇的是，集成的错误率比最好的个体还低

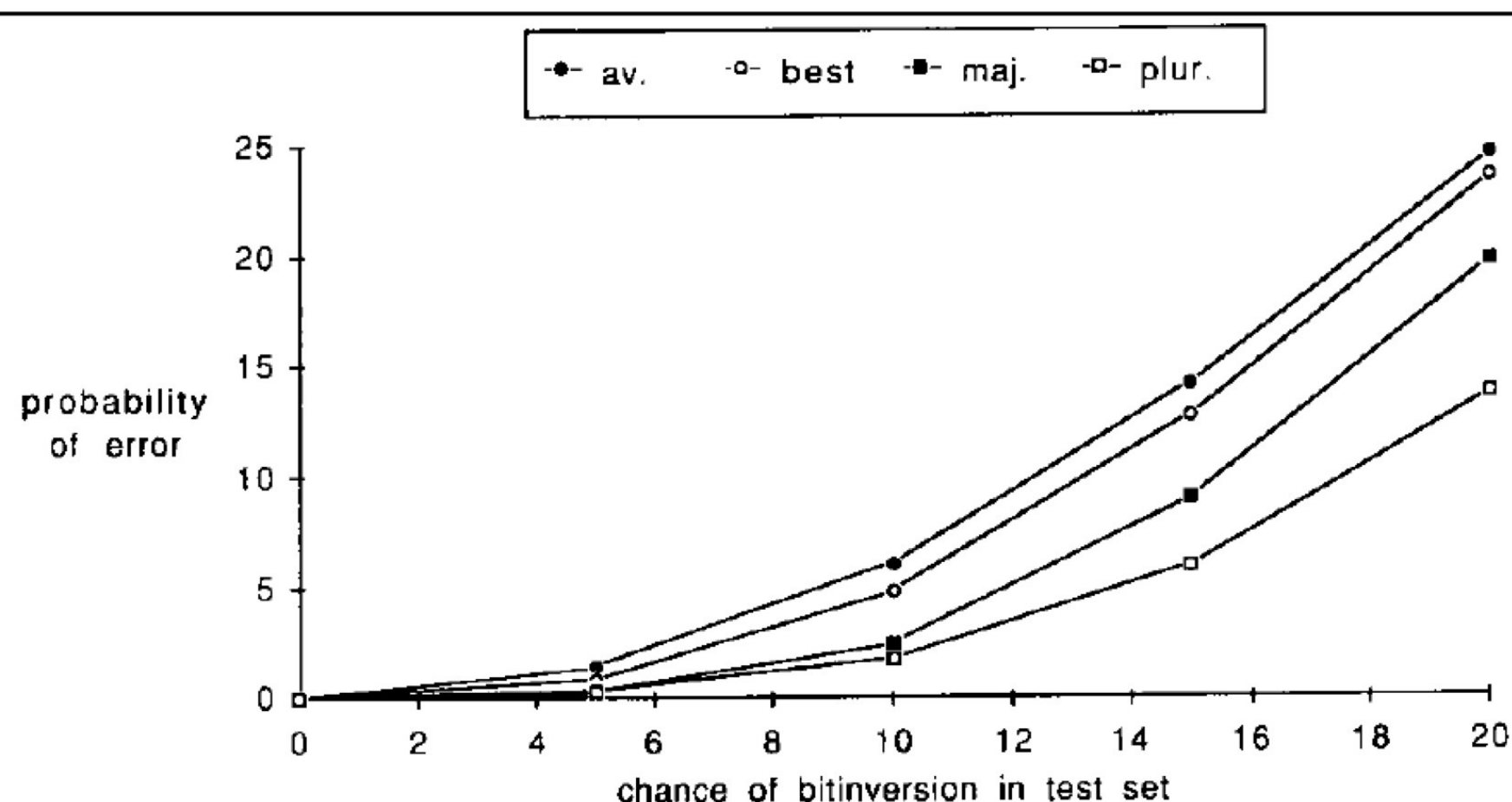


Fig. 4. Performance versus noise level in the test set is shown for individual and for consensus decisions. Data displayed shows the average and the best network, as well as collective decisions using majority and plurality for seven networks trained on individual training sets.

[L.K. Hansen & P. Salamon, TPAMI90]

由于集成学习技术可以有效地提高学习系统的泛化能力，因此它成为国际机器学习界的研究热点，并被国际权威 T.G. Dietterich 称为当前机器学习四大研究方向之首
[T.G. Dietterich, AIMag97]

集成学习的应用

集成学习技术已经在行星探测、地震波分析、Web信息过滤、生物特征识别、计算机辅助医疗诊断等众多领域得到了广泛的应用

只要能用到机器学习的地方，就能用到集成学习

如何构建好的集成

集成包含 T 个基学习器 $\{h_1, h_2, \dots, h_T\}$

	测试例1	测试例2	测试例3
h_1	✓	✓	✗
h_2	✗	✓	✓
h_3	✓	✗	✓
集成	✓	✓	✓

(a) 集成提升性能

	测试例1	测试例2	测试例3
h_1	✓	✓	✗
h_2	✓	✓	✗
h_3	✓	✓	✗
集成	✓	✓	✗

(b) 集成不起作用

	测试例1	测试例2	测试例3
h_1	✓	✗	✗
h_2	✗	✓	✗
h_3	✗	✗	✓
集成	✗	✗	✗

(c) 集成起负作用

个体学习器越精确、差异越大，集成越好

集成个体应该 **“好而不同”**

如何做到好而不同

- 针对输入数据：使用采样的方法得到不同的样本
- 针对算法本身：
 - 个体学习器来自不同的模型集合（逻辑回归/决策树）
 - 个体学习器来自于同一个模型集合的不同超参数，例如学习率 η 不同
 - 算法本身具有随机性，例如用不同的随机种子来得到不同的模型

个体越多越好吗？

既然多个个体的集成比单个个体更好，那么是不是个体越多越好？

更多的个体意味着：

- 在预测时需要更大的计算开销，因为要计算更多的个体预测
- 更大的存储开销，因为有更多的个体需要保存

个体的增加将使得个体间的差异越来越难以获得

集成学习：如何构造？

• 平均法

对于数值类的回归预测问题，通常使用的结合策略是平均法，即对于若干个弱学习器的输出进行平均得到最终的预测输出。最简单的平均是算术平均，最终预测为：

$$H(x) = \frac{1}{T} \sum_{i=1}^T h_i(x)$$

如果每个学习器有一个权重，则采用加权平均，最终预测为：

$$H(x) = \frac{1}{T} \sum_{i=1}^T w_i h_i(x)$$

其中 w_i 是个体学习器 h_i 的权重，有 $w_i \geq 0$ ， $\sum_{i=1}^T w_i = 1$

一般而言，在个体学习器性能相差较大时宜使用加权平均法，而在个体学习器性能相近时宜使用算术平均法。

集成学习：如何构造？

- **投票法**

假设预测类别是： $\{c_1, c_2, \dots, c_k\}$

有T个弱学习器，对应T个分类结果。

数量最多的类别为最终的分类类别。如果不止一个类别获得最高票，则随机选择一个做最终类别。

集成学习：三种方法

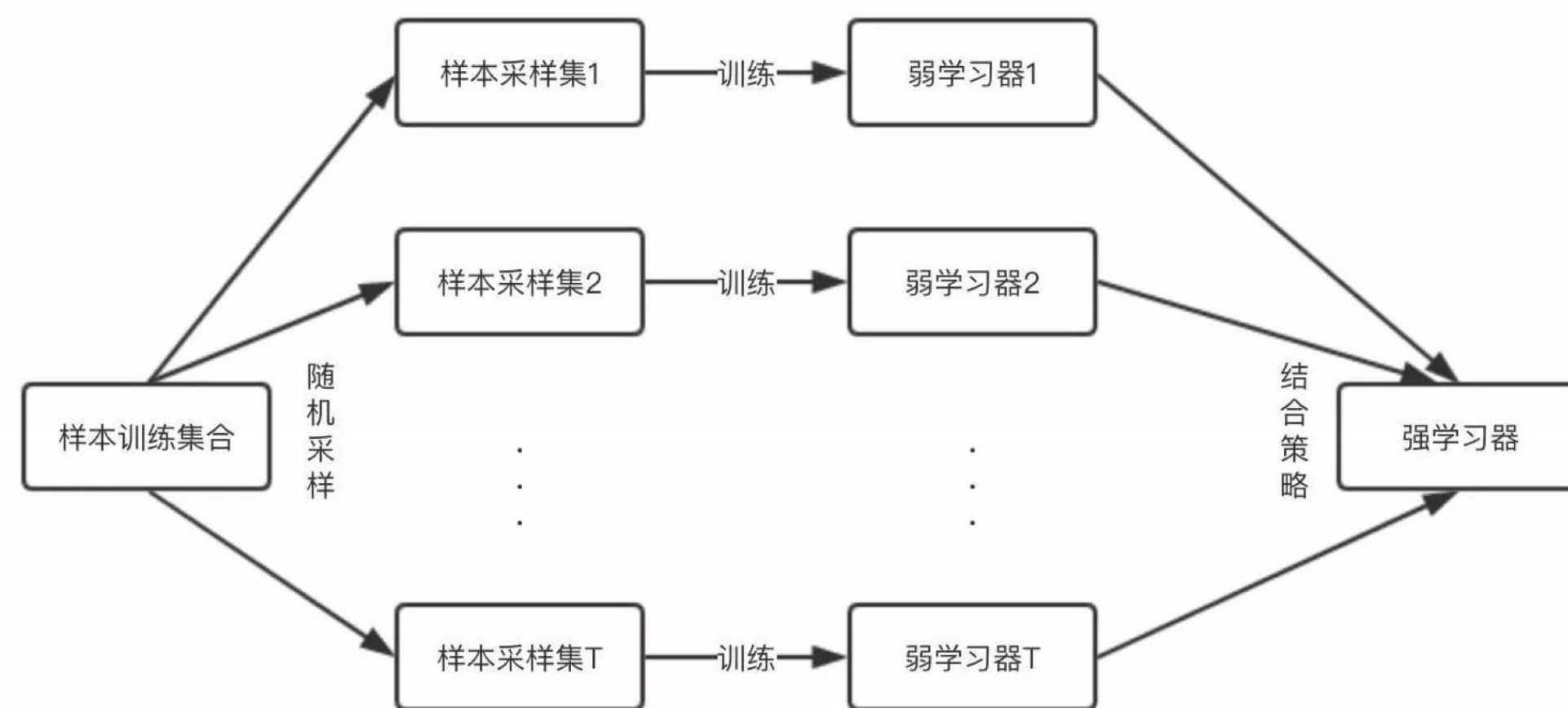
- Bagging (Bootstrap Aggregation)
- Random Forest (也是Bagging的一种)
- Boosting

集成学习：Bagging

民主

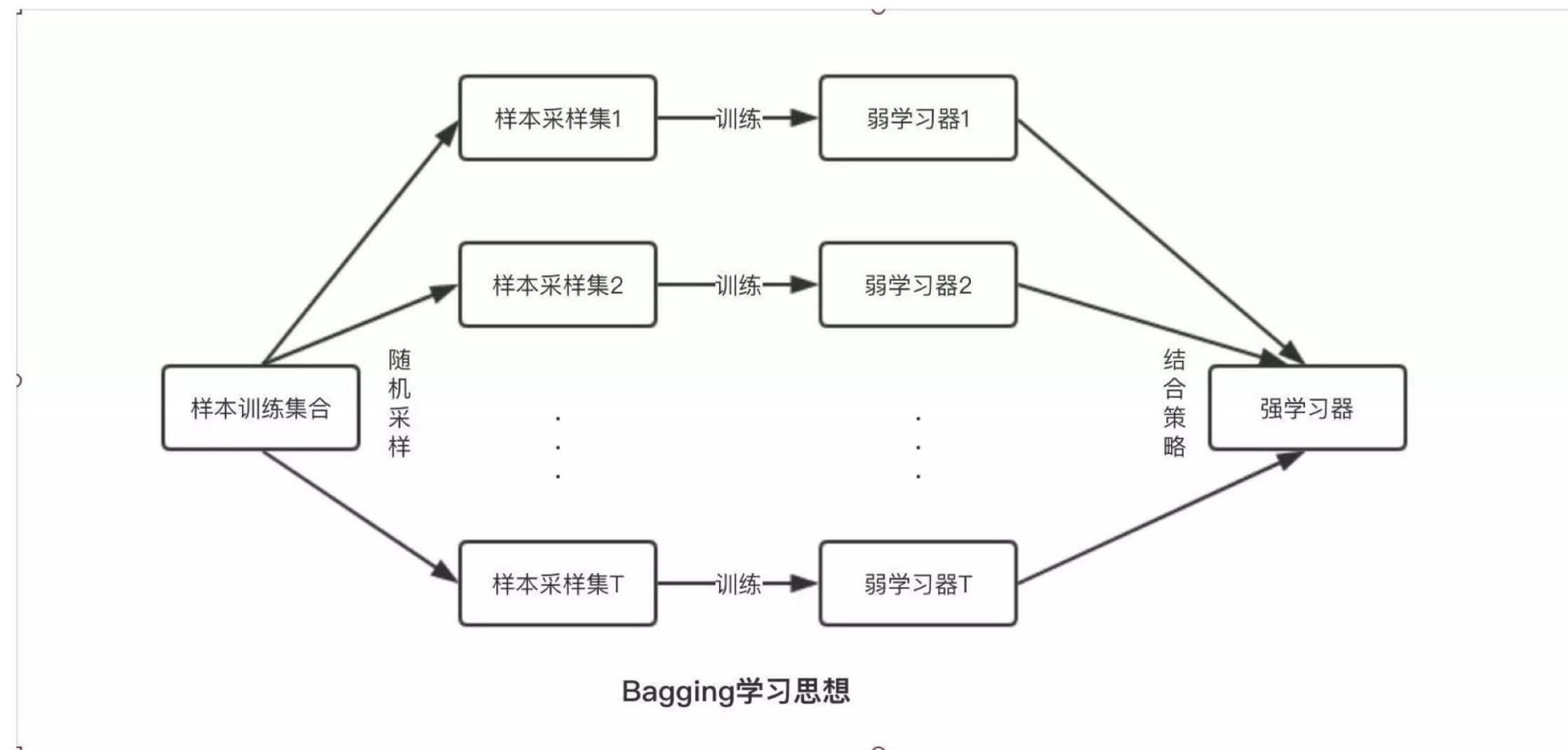
Bagging 的核心思路

Bagging 的核心思路是——民主。
Bagging 的思路是所有基础模型都一致对待，每个基础模型手里都只有一票。然后使用民主投票的方式得到最终的结果。



Bagging学习思想

集成学习：Bagging



具体过程：

- 1.从原始样本训练集中采样训练集。每轮从原始样本集中使用Bootstrap的方法抽取N个训练样本（在训练集中，有些样本可能被多次抽取到，而有些样本可能一次都没有被抽中）。共进行k轮抽取，得到k个训练集。（k个训练集之间是相互独立的）
- 2.每次使用一个训练集得到一个弱学习器模型，k个训练集共得到k个弱学习器模型。（注：这里并没有具体的分类算法或回归方法，我们可以根据具体问题采用不同的分类或回归方法，如决策树、感知器等）
- 3.对分类问题：将上步得到的k个模型采用投票的方式得到分类结果；对回归问题，计算上述模型的均值作为最后的结果。（所有模型的重要性相同）

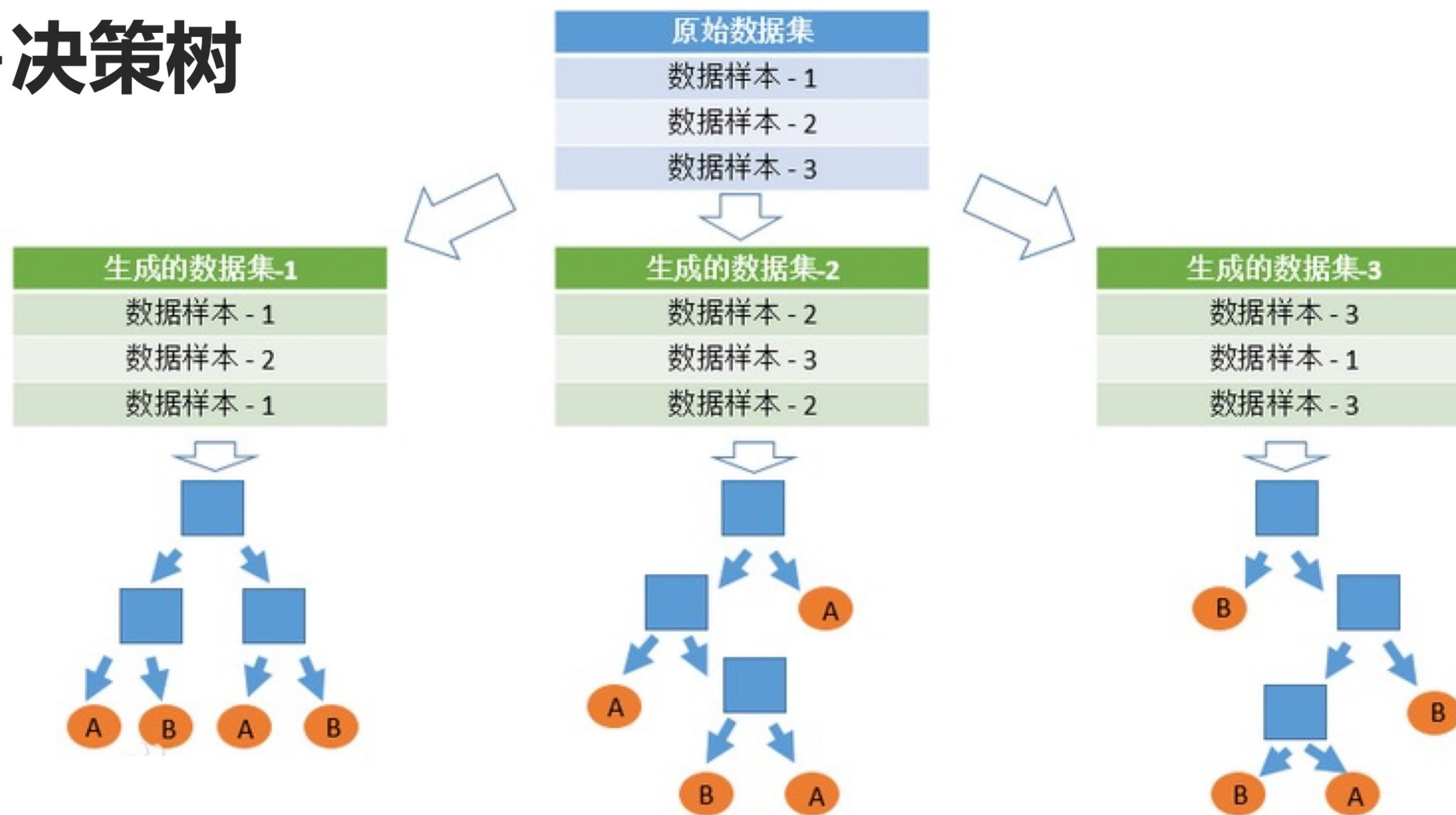
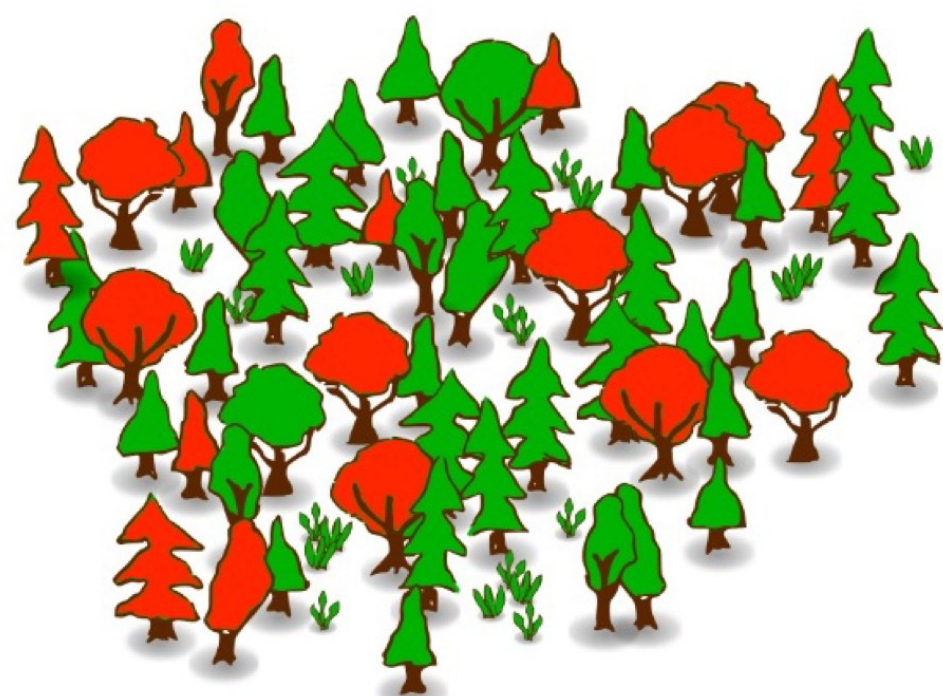
集成学习：Bagging

自助采样（Bootstrap）法

- Bootstrap是一种从给定训练集中**有放回的均匀抽样**，也就是说，每当选中一个样本，它等可能地被再次选中并被再次添加到训练集中。
- 给定包含N个样本的数据集，我们先随机取出一个样本放入采样集中，再把该样本放回初始数据集，使得下次采样时该样本仍有可能被选中。
- 经过N轮随机采样，我们得到N个样本的采样集，初始训练集中有的样本在采样集中多次出现，有的则从未出现。

集成学习：Bagging

随机森林=Bagging+决策树



- 随机森林由很多决策树构成，不同决策树之间没有关联。
- 当进行分类任务时，新的输入样本进入，让森林中的每一棵决策树分别进行分类，每个决策树会得到一个自己的分类结果，决策树的分类结果中哪一个分类最多，那么随机森林就会把这个结果当做最终的结果。

集成学习：Bagging

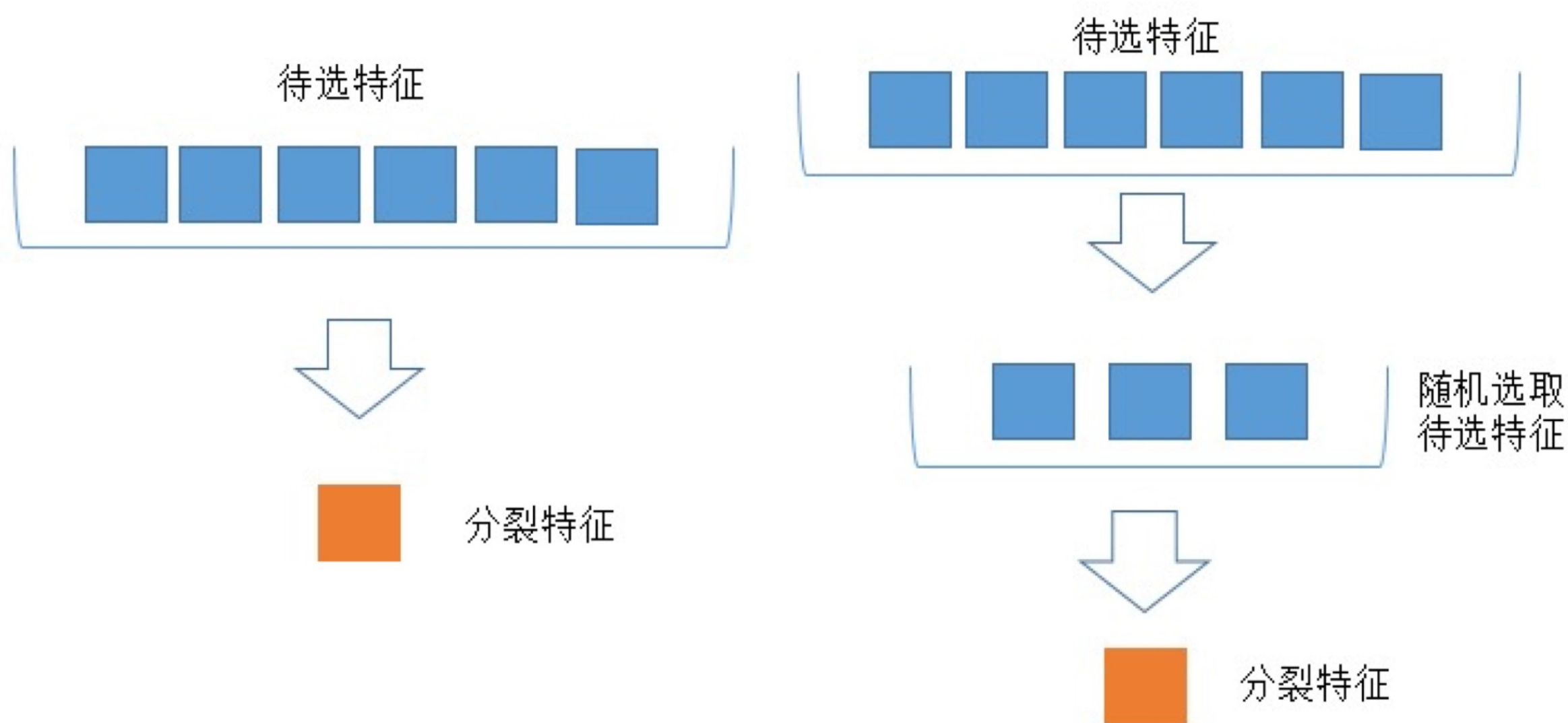
随机森林——构造随机森林的 4 个步骤



1. 一个样本容量为 N 的样本，有放回的抽取 N 次，每次抽取1个，最终形成了 N 个样本。这选择好了的 N 个样本用来训练一个决策树，作为决策树根节点处的样本。
2. 假设每个样本有 M 个属性，在决策树的每个节点需要分裂时，随机从这 M 个属性中选取 m 个属性，满足条件 $m < M$ 。然后从这 m 个属性中采用某种策略（如信息增益）来选择1个属性作为该节点的分裂属性。（下一页PPT图释）
3. 决策树形成过程中每个节点都要按照步骤2来分裂。一直到不能够再分裂为止。
4. 按照步骤1~3建立大量的决策树，这样就构成了随机森林了。

集成学习：Bagging

随机森林



决策树选取分裂特征过程

随机森林子树选取分裂特征过程

蓝色的方块代表所有可以被选择的特征，也就是待选特征。黄色的方块是分裂特征。左边是一棵决策树的特征选取过程，通过在待选特征中选取最优的分裂特征（采用信息增益或者Gini指数作为分裂策略），完成分裂。右边是一个随机森林中的子树的特征选取过程。

集成学习：Bagging

随机森林特点：

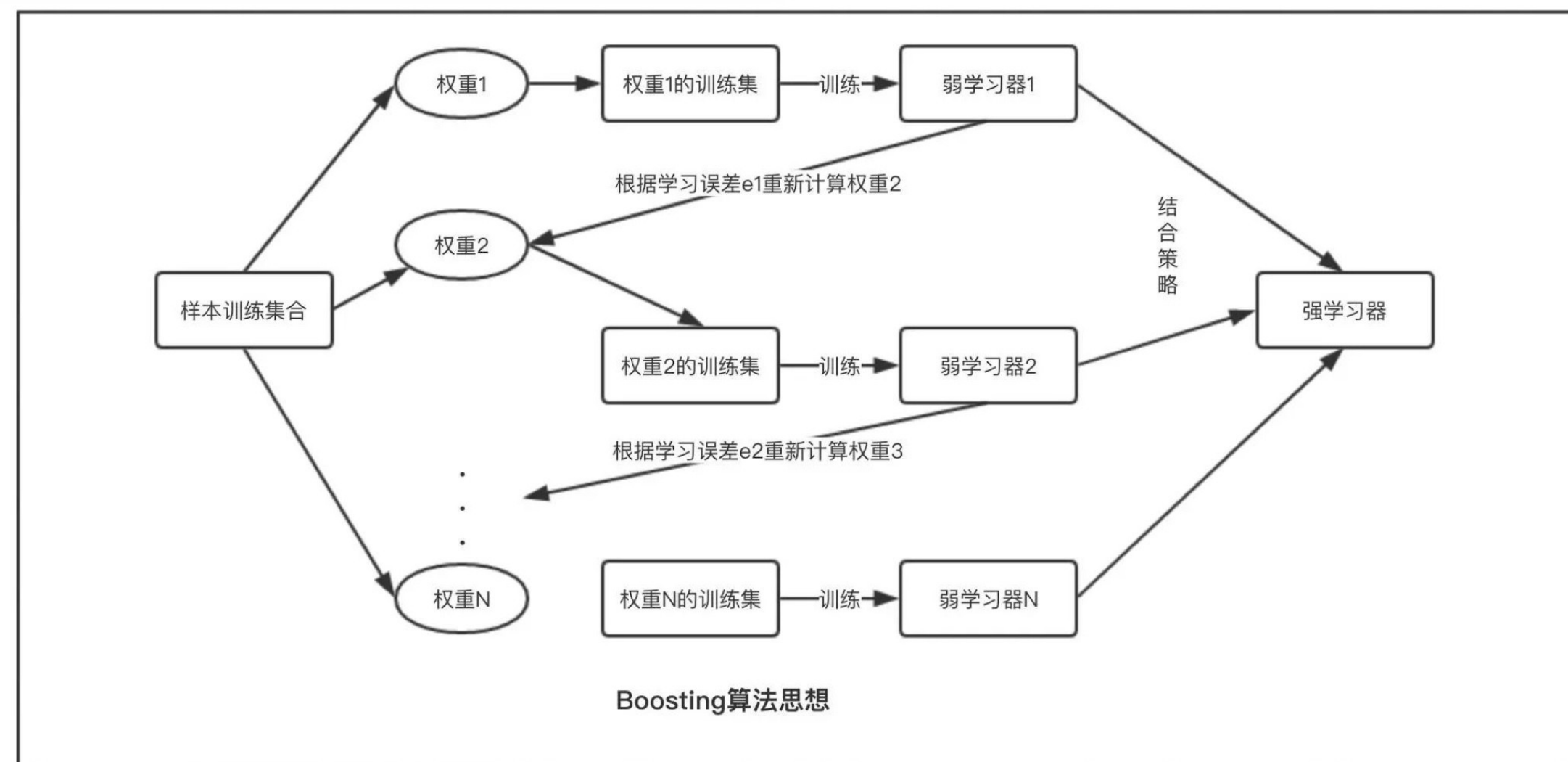
- 个体学习器为决策树
- 对训练样本进行采样
- 对属性进行随机采样
- 训练可以高度并行化
- 由于对决策树候选划分属性的采样，在样本特征维度较高时，仍可以高效的训练模型
- 有了样本和属性的采样，最终训练出来的模型泛化能力强。

集成学习：Boosting

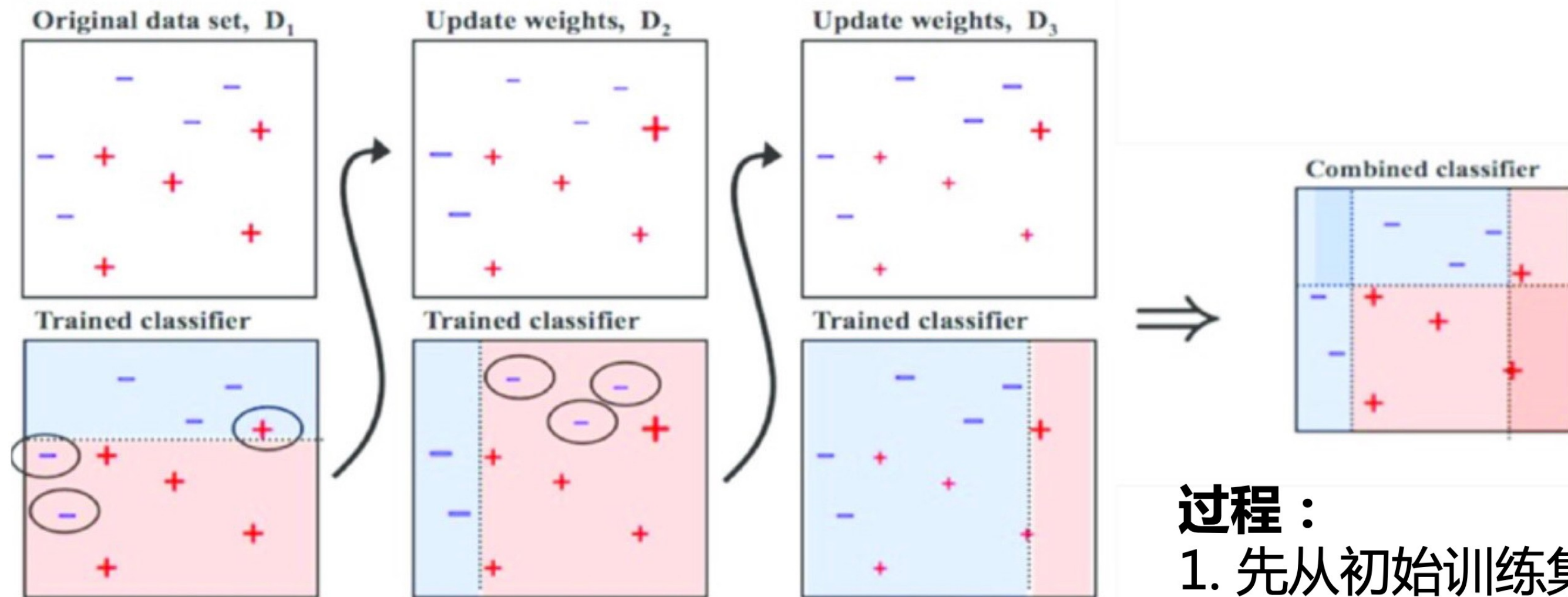
挑选精英

Boosting 的核心思路

Boosting 的核心思路是 —— 挑选精英。
Boosting 和 bagging 最本质的差别在于它对基础模型不是一致对待的，而是经过不停的考验和筛选来挑选出【精英】，然后给精英更多的投票权，表现不好的基础模型则给较少的投票权，最后综合所有模型的投票得到最终结果。



集成学习：Boosting



第 t 个学习器的误差： ϵ_t

$$\text{第}t\text{个学习器的权重} : \beta_m = \begin{cases} \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right), & \text{if } \epsilon_t \leq 0.5 \\ 0, & \text{else} \end{cases}$$

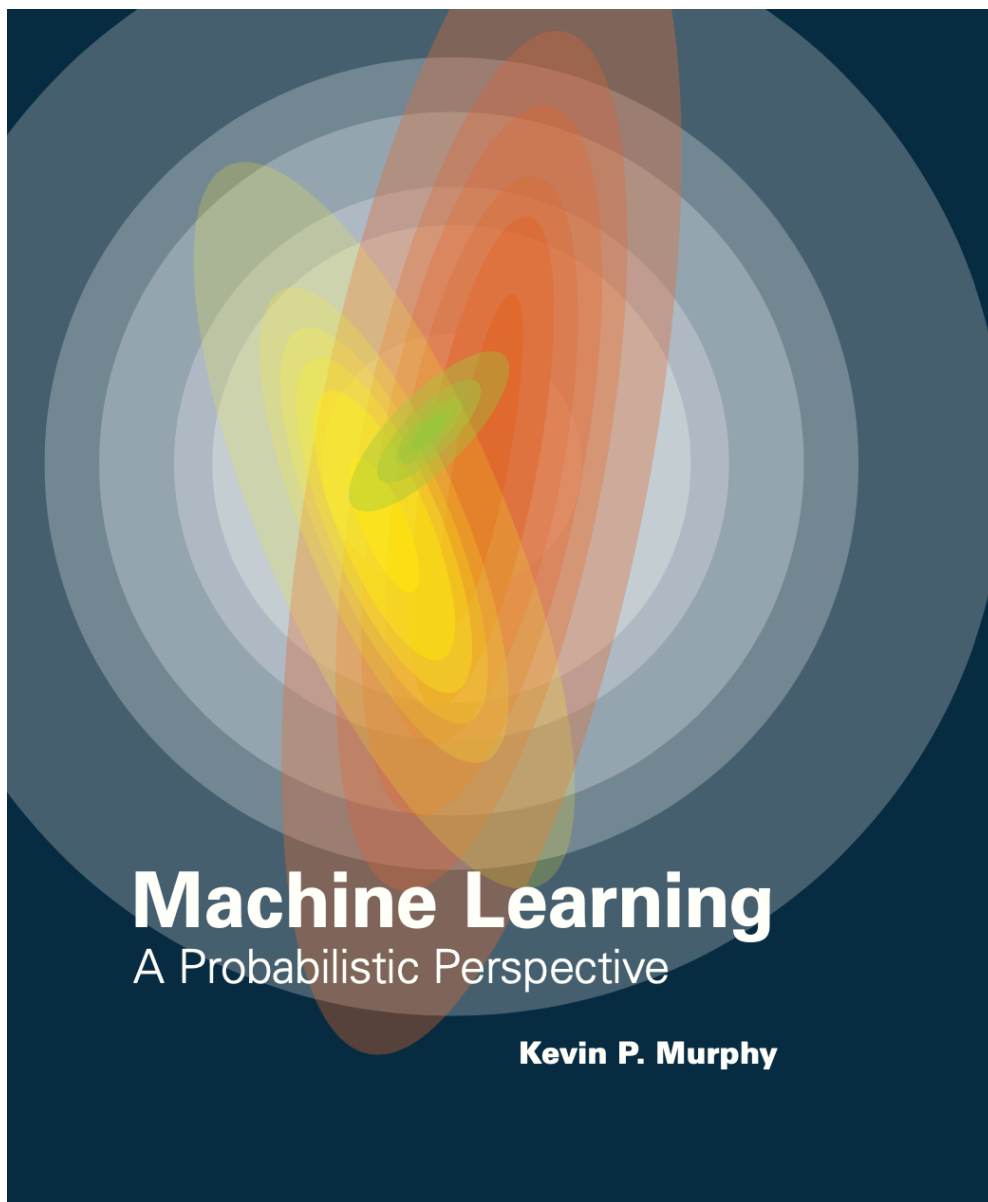
第 $t+1$ 轮数据集的权重：

$$w_{i,m+1} = w_{i,m} \times \begin{cases} \exp(-\beta_m), & \text{如果样本分类正确} \\ \exp(\beta_m), & \text{如果样本分类错误} \end{cases}$$

过程：

AdaBoost算法：

1. 先从初始训练集训练一个弱学习器。
2. 根据误差率更新训练样本的权重，使得误差率高的样本在训练集中得到更多的重视（给予更多权重），误差率低的样本给予更少权重，再训练一个新的弱学习器。
3. 重复第2步，直到弱学习器达到事先指定的数目 T ，最终将这 T 个弱学习器通过集合策略进行整合，得到最终的强学习器。
4. 对于最终的结果，如果是分类任务，按照权重进行投票，如果是回归任务，进行加权，再进行预测。



Chapter 16: Adaptive basis function models

Adaptive basis function model

$$f(\mathbf{x}) = w_0 + \sum_{m=1}^M w_m \phi_m(\mathbf{x})$$

You choose ϕ_m and w_m

The goal of boosting is to solve

$$\min_f \sum_{i=1}^N L(y_i, f(\mathbf{x}_i))$$

You choose L

If we use squared error loss, the optimal estimate is given by

$$f^*(\mathbf{x}) = \operatorname{argmin}_{f(\mathbf{x})} \mathbb{E}_{y|\mathbf{x}} [(Y - f(\mathbf{x}))^2] = \mathbb{E}[Y|\mathbf{x}]$$

Name	Loss	Derivative	f^*	Algorithm
Squared error	$\frac{1}{2}(y_i - f(\mathbf{x}_i))^2$	$y_i - f(\mathbf{x}_i)$	$\mathbb{E}[y \mathbf{x}_i]$	L2Boosting
Absolute error	$ y_i - f(\mathbf{x}_i) $	$\text{sgn}(y_i - f(\mathbf{x}_i))$	$\text{median}(y \mathbf{x}_i)$	Gradient boosting
Exponential loss	$\exp(-\tilde{y}_i f(\mathbf{x}_i))$	$-\tilde{y}_i \exp(-\tilde{y}_i f(\mathbf{x}_i))$	$\frac{1}{2} \log \frac{\pi_i}{1-\pi_i}$	AdaBoost
Logloss	$\log(1 + e^{-\tilde{y}_i f_i})$	$y_i - \pi_i$	$\frac{1}{2} \log \frac{\pi_i}{1-\pi_i}$	LogitBoost

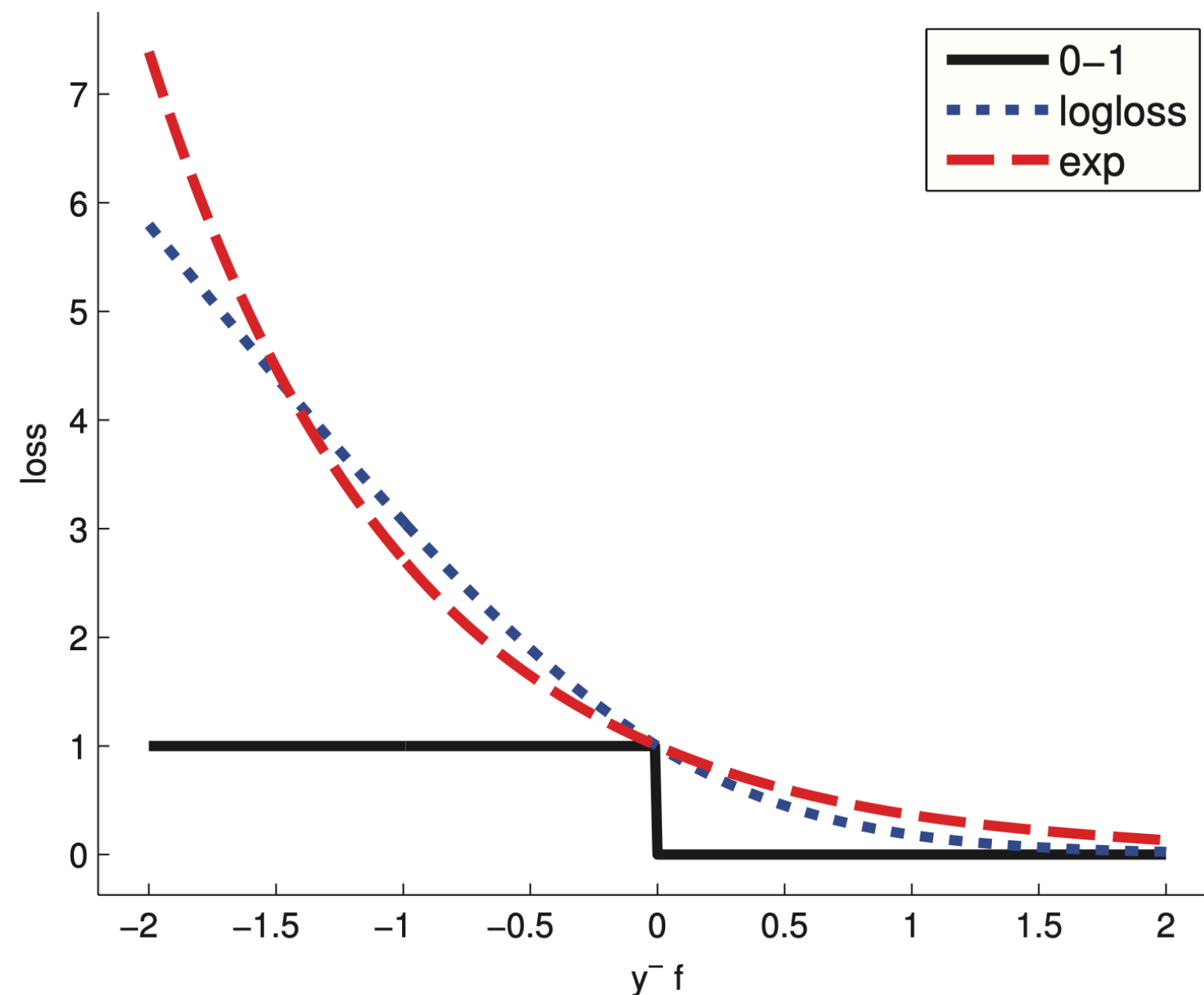
Table 16.1 Some commonly used loss functions, their gradients, their population minimizers f^* , and some algorithms to minimize the loss. For binary classification problems, we assume $\tilde{y}_i \in \{-1, +1\}$, $y_i \in \{0, 1\}$ and $\pi_i = \text{sigm}(2f(\mathbf{x}_i))$. For regression problems, we assume $y_i \in \mathbb{R}$. Adapted from (Hastie et al. 2009, p360) and (Buhlmann and Hothorn 2007, p483). $\pi_i = \mathbb{P}[y = 1 \mid X = x_i]$

$$\begin{aligned}
& \min_f \mathbb{E}_{Y|X}(y - f(x))^2 && \text{Denote } \mu(x) = \mathbb{E}[y \mid x] \\
& (y - f(x))^2 = (y - \mu(x) + \mu(x) - f(x))^2 \\
& \quad = (y - \mu(x))^2 + (\mu(x) - f(x))^2 + 2(y - \mu(x))(\mu(x) - f(x)) \\
& \implies f^*(x) = \mu(x)
\end{aligned}$$

Adaboost

Name	Loss	Derivative	f^*	Algorithm
Exponential loss	$\exp(-\tilde{y}_i f(\mathbf{x}_i))$	$-\tilde{y}_i \exp(-\tilde{y}_i f(\mathbf{x}_i))$	$\frac{1}{2} \log \frac{\pi_i}{1-\pi_i}$	AdaBoost

$$f^*(\mathbf{x}) = \frac{1}{2} \log \frac{p(\tilde{y} = 1|\mathbf{x})}{p(\tilde{y} = -1|\mathbf{x})}$$



But we want to choose f^* from the following $\mathcal{H}$

$$f(\mathbf{x}) = w_0 + \sum_{m=1}^M w_m \phi_m(\mathbf{x})$$

Adaboost

$$f(\mathbf{x}) = w_0 + \sum_{m=1}^M w_m \phi_m(\mathbf{x})$$

Then at iteration m , we compute

Forward statewide additive modeling

$$(\beta_m, \gamma_m) = \operatorname{argmin}_{\beta, \gamma} \sum_{i=1}^N L(y_i, f_{m-1}(\mathbf{x}_i) + \beta \phi(\mathbf{x}_i; \gamma))$$

and then we set

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m \phi(\mathbf{x}; \gamma_m)$$

At step m we have to minimize

$$L_m(\phi) = \sum_{i=1}^N \exp[-\tilde{y}_i(f_{m-1}(\mathbf{x}_i) + \beta \phi(\mathbf{x}_i))] = \sum_{i=1}^N w_{i,m} \exp(-\beta \tilde{y}_i \phi(\mathbf{x}_i))$$

Here, $w_{i,m} \triangleq \exp(-\tilde{y}_i f_{m-1}(\mathbf{x}_i))$
We will later derive a recursive form of it

Adaboost

At step m we have to minimize

$$L_m(\phi) = \sum_{i=1}^N \exp[-\tilde{y}_i(f_{m-1}(\mathbf{x}_i) + \beta\phi(\mathbf{x}_i))] = \sum_{i=1}^N w_{i,m} \exp(-\beta\tilde{y}_i\phi(\mathbf{x}_i))$$

$$\begin{aligned} L_m &= e^{-\beta} \sum_{\tilde{y}_i = \phi(\mathbf{x}_i)} w_{i,m} + e^{\beta} \sum_{\tilde{y}_i \neq \phi(\mathbf{x}_i)} w_{i,m} \\ &= (e^{\beta} - e^{-\beta}) \sum_{i=1}^N w_{i,m} \mathbb{I}(\tilde{y}_i \neq \phi(\mathbf{x}_i)) + e^{-\beta} \sum_{i=1}^N w_{i,m} \end{aligned}$$

Consequently the optimal function to add is

$$\phi_m = \operatorname{argmin}_{\phi} w_{i,m} \mathbb{I}(\tilde{y}_i \neq \phi(\mathbf{x}_i))$$

This is to train ϕ to fit the weighted data

Adaboost

$$L_m = e^{-\beta} \sum_{\tilde{y}_i = \phi(\mathbf{x}_i)} w_{i,m} + e^{\beta} \sum_{\tilde{y}_i \neq \phi(\mathbf{x}_i)} w_{i,m}$$

Substituting ϕ_m into L_m and solving for β we find

$$\beta_m = \frac{1}{2} \log \frac{1 - \text{err}_m}{\text{err}_m} \quad \text{err}_m = \frac{\sum_{i=1}^N w_i \mathbb{I}(\tilde{y}_i \neq \phi_m(\mathbf{x}_i))}{\sum_{i=1}^N w_{i,m}}$$

The overall update becomes

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m \phi_m(\mathbf{x})$$

With this, the weights at the next iteration become

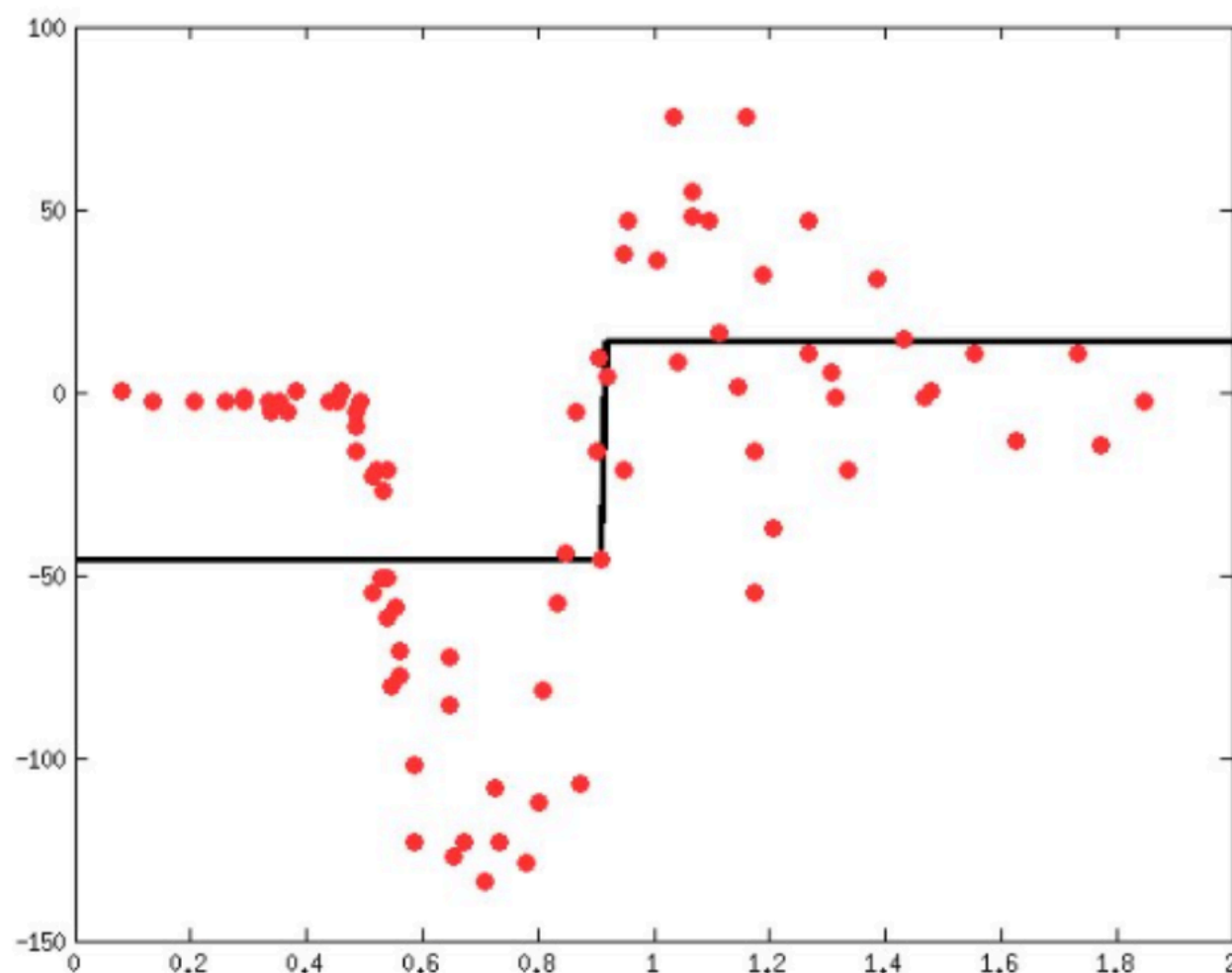
$$\begin{aligned} w_{i,m+1} &= w_{i,m} e^{-\beta_m \tilde{y}_i \phi_m(\mathbf{x}_i)} \\ &= w_{i,m} e^{\beta_m (2\mathbb{I}(\tilde{y}_i \neq \phi_m(\mathbf{x}_i)) - 1)} \\ &= w_{i,m} e^{2\beta_m \mathbb{I}(\tilde{y}_i \neq \phi_m(\mathbf{x}_i))} e^{-\beta_m} \end{aligned}$$

如果判对的数据点 $e^{-\beta_m}$;
如果判错的数据点 e^{β_m}

对于回归则是不停的拟合残差

- ▶ Learn a regression predictor
- ▶ Compute the error residual
- ▶ Learn to predict the residual

Learn a simple predictor...

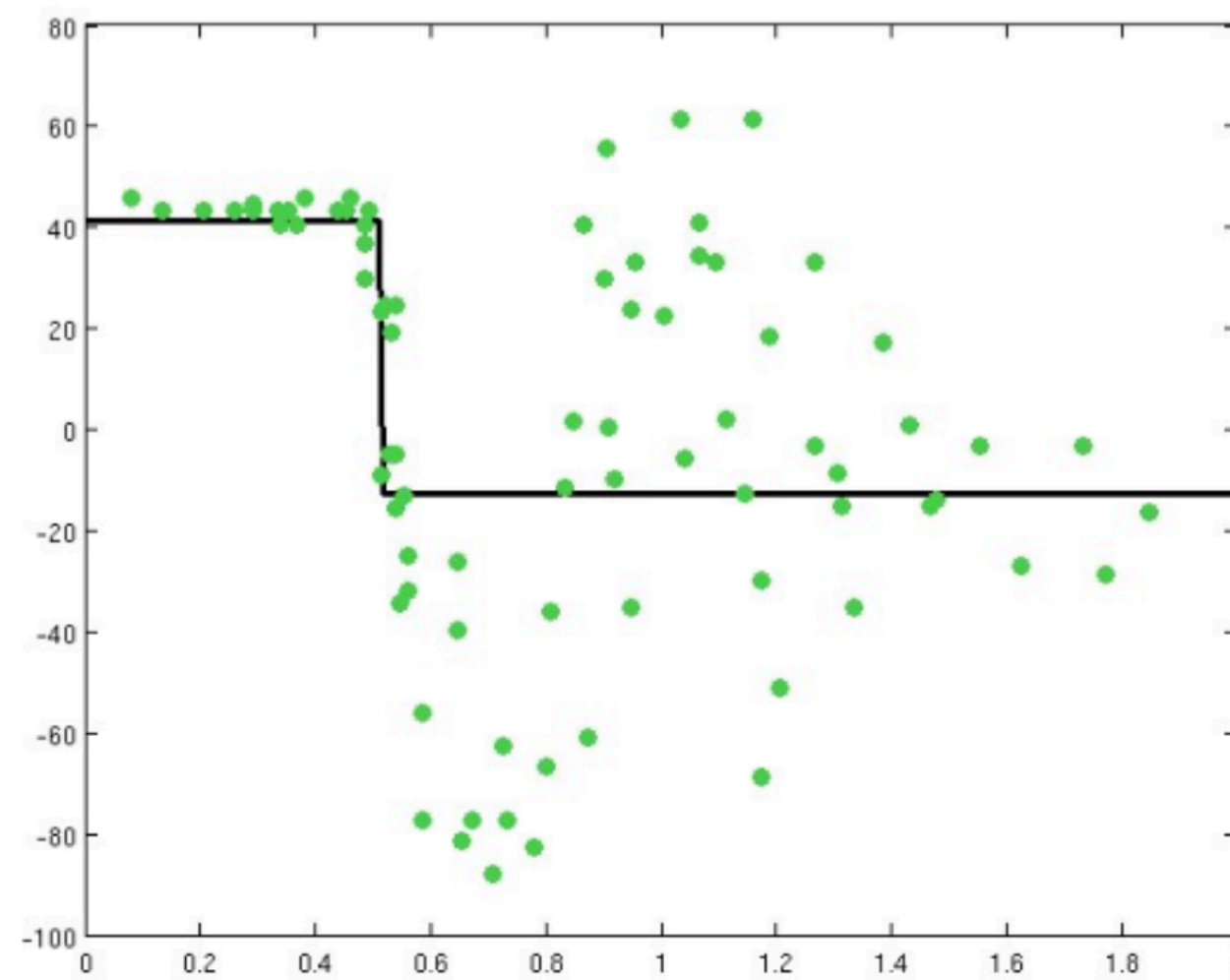


L2boosting

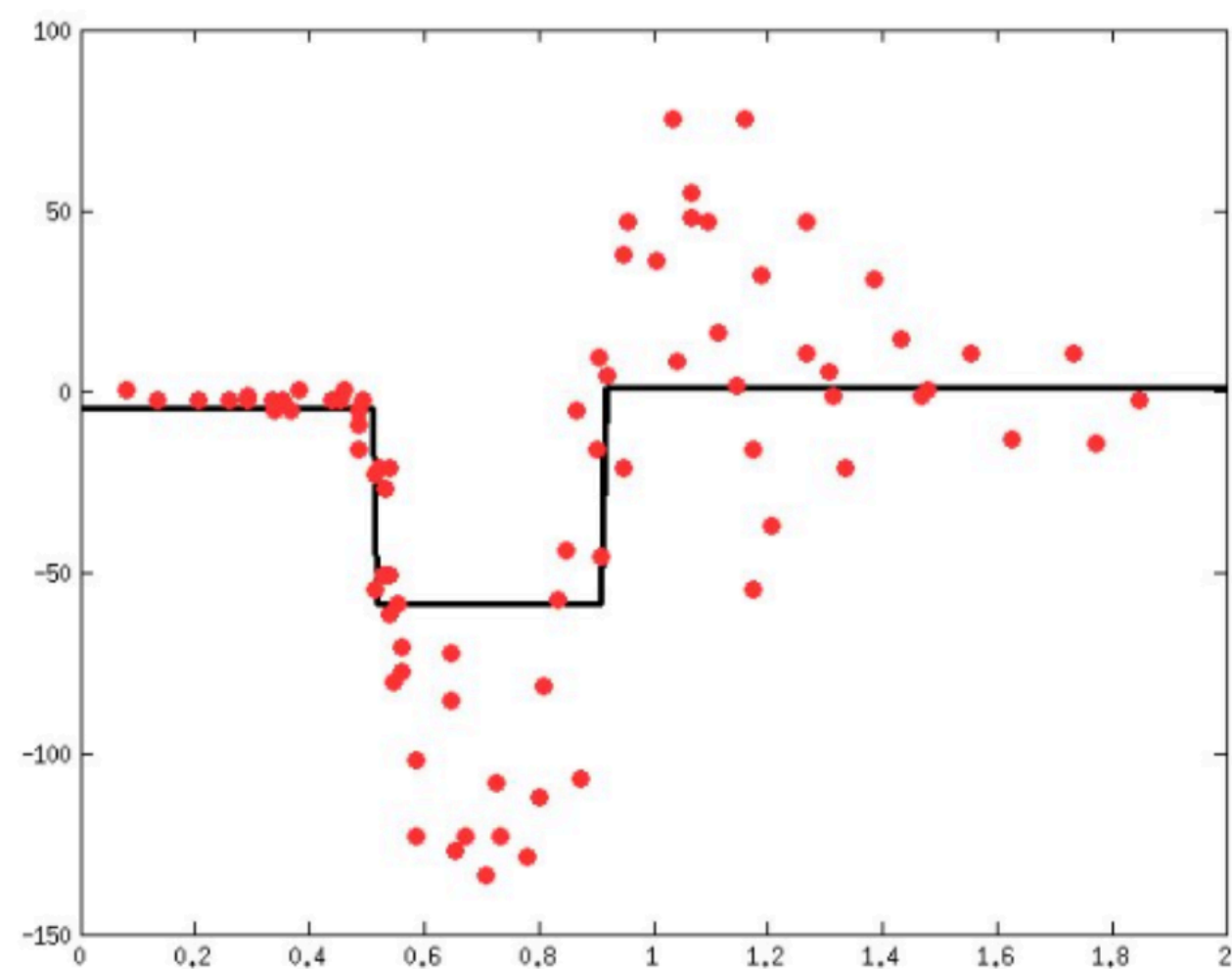
Suppose we used squared error loss. Then at step m the loss

$$L(y_i, f_{m-1}(\mathbf{x}_i) + \beta\phi(\mathbf{x}_i; \gamma)) = (r_{im} - \phi(\mathbf{x}_i; \gamma))^2$$

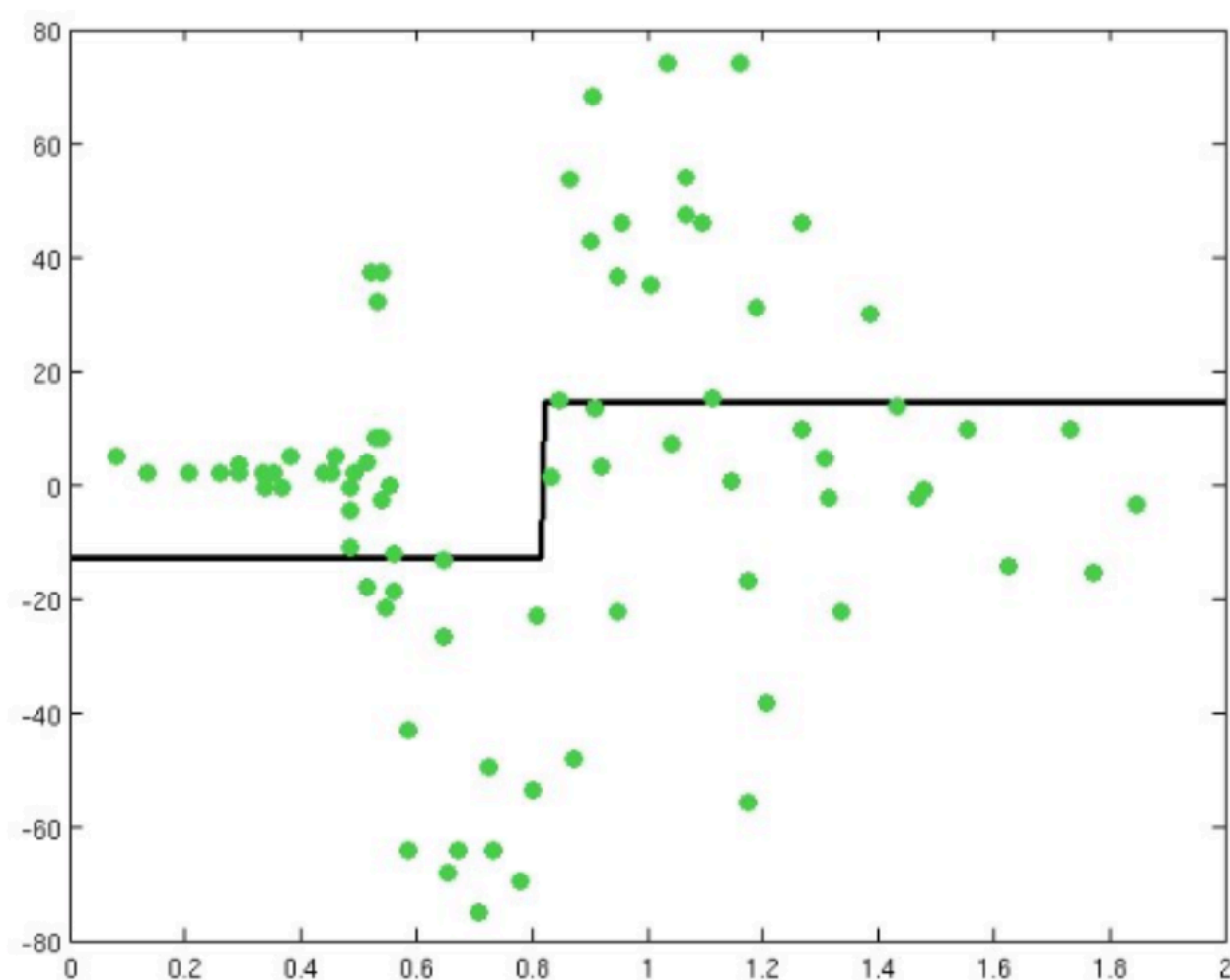
Then try to correct its errors



Combining gives a better predictor...

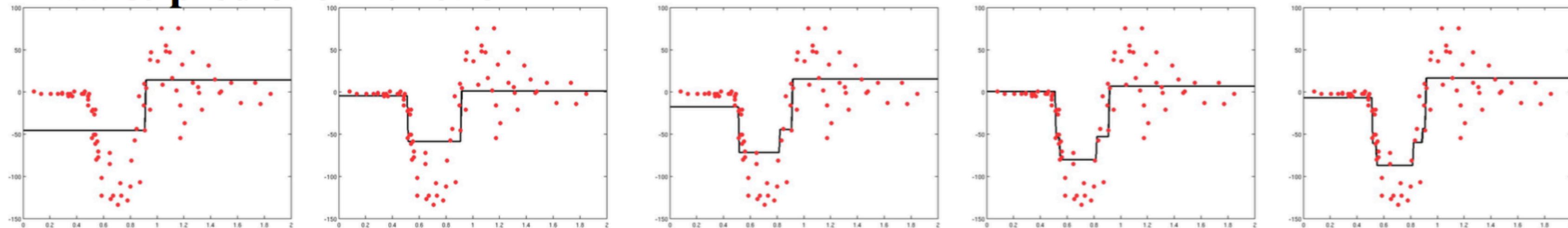


Can try to correct its errors also, & repeat

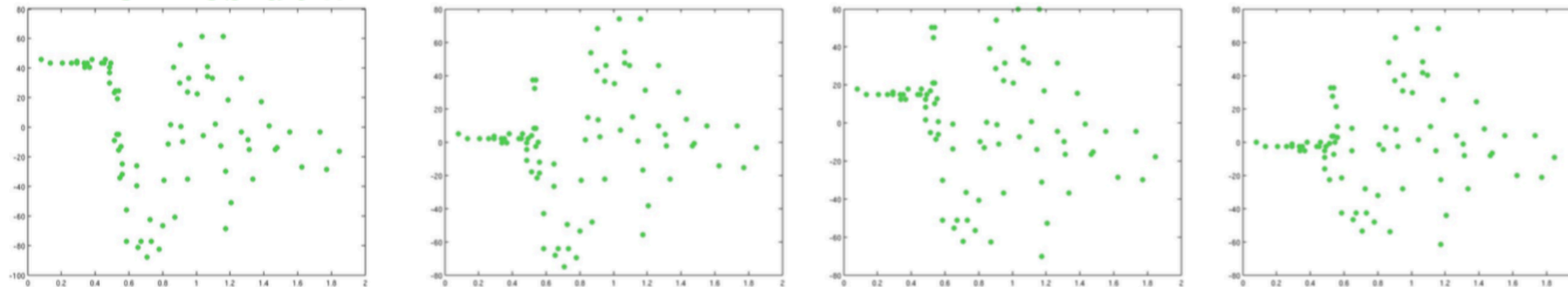


- ▶ Learn a regression predictor
- ▶ Sum of predictions is increasingly accurate
- ▶ Predictive function is increasingly complex

Data & prediction function



Error residual



...

集成学习：Bagging 和 Boosting的差别

Bagging:

样本选择

训练集是在原始集中有放回选取的，从原始集中选出的各轮训练集之间是独立的。

样例权重

使用均匀取样，每个样例的权重相等。

预测函数

所有预测函数的权重相等。

并行计算

各个预测函数可以并行生成。

Boosting:

每一轮的训练集不变，只是训练集中每个样例在分类器中的权重发生变化。而权值是根据上一轮的分类结果进行调整。

根据错误率不断调整样例的权值，错误率越大则权重越大。

每个弱分类器都有相应的权重，对于分类误差小的分类器会有更大的权重。

各个预测函数只能顺序生成，因为后一个模型参数需要前一轮模型的结果。